GETTING STARTED WITH REST APIs

A Beginner's Guide using JSONPlaceholder & OpenWeather API

Technology: REST APIs (HTTP GET & POST Methods)
Platform: Linux | Tools: VS Code, Postman, cURL
March 2026

Section 1: Title & Objective

1.1 Technology Chosen

REST APIs (Representational State Transfer Application Programming Interfaces), explored using two platforms: JSONPlaceholder for safe, sandboxed practice and the OpenWeather API for real-world authenticated requests.

1.2 Why This Technology?

REST APIs are the backbone of modern software communication. Nearly every application — from mobile banking to weather apps — relies on APIs to send and receive data. Understanding how they work is a highly practical skill applicable in technical support, QA, product operations, and beyond. It does not require deep coding knowledge, making it an ideal entry point for anyone working in a technical environment.

JSONPlaceholder was chosen as the learning platform because it requires no sign-up, no API key, and no setup — requests can be made immediately. OpenWeather was selected to introduce real-world concepts: authentication, API keys, and query parameters.

1.3 End Goal

By the end of this guide, the reader will be able to:

- Understand what a REST API is and how it works.
- Send GET and POST requests using Postman and the cURL terminal tool on Linux.
- Read and interpret JSON responses from a server.
- Authenticate a request to a live API using an API key.
- Diagnose common API errors using HTTP status codes.

> **Note:** *This guide is written for Linux users with some familiarity with terminals. All commands use the bash shell. No prior API experience is required.*

Section 2: Quick Summary of the Technology

2.1 What Is a REST API?

A REST API is a standardised way for two software systems to communicate over the internet using HTTP — the same protocol your browser uses to load web pages. REST stands for Representational State Transfer, and it defines a set of rules for how requests are structured and how servers should respond.

Think of a REST API like a waiter in a restaurant: your application (the client) places an order with the waiter (the API), who takes it to the kitchen (the server) and brings back exactly what was requested — no more, no less.

2.2 Where Is It Used?

REST APIs are used everywhere in modern technology:

- Weather apps on your phone fetch live data from a weather API.
- Ride-hailing apps communicate driver locations via APIs.
- Online checkouts process payments through payment gateway APIs.
- Social media platforms allow third-party apps to post, read, and like content through APIs.

2.3 Core Concepts at a Glance

Term	Plain-English Meaning
Endpoint	A specific URL that represents a resource, e.g. /users/1 to fetch users with ID 1.
HTTP Method	The action to perform: GET (read), POST (create), PUT (update), DELETE (remove).
JSON	JavaScript Object Notation — the standard data format APIs use to send and receive data.
Status Code	A 3-digit number indicating the result: 200 = success, 404 = not found, 401 = unauthorised.
API Key	A unique token included in requests to identify and authenticate the calling application.
Query Param	Extra filters appended to the URL after a ?, e.g. ?units=metric&q=Nairobi.

2.4 Real-World Example

The OpenWeather API is used by millions of applications globally. When you open a weather app on your phone and see the current temperature in Nairobi, that data was almost certainly retrieved via a GET request to a weather API — authenticated by an API key and filtered by a city name query parameter. This guide replicates that exact interaction from scratch.

Section 3: System Requirements

The following tools and configurations are required before proceeding to installation. All are free and available on Linux.

3.1 Operating System

This guide is written for Linux (any major distribution: Ubuntu, Debian, Fedora, Arch). The terminal commands use bash syntax. Windows users may use WSL2 (Windows Subsystem for Linux) or Git Bash as alternatives.

3.2 Required Tools

Tool	Purpose & Notes
VS Code	The recommended code editor for viewing and editing JSON responses and saving notes. Download from code.visualstudio.com.
cURL	A command-line tool for making HTTP requests directly from the terminal. Usually pre-installed on Linux — verify with: curl --version
Postman	A graphical API testing tool for constructing and inspecting requests without writing code. Download from postman.com.
Web Browser	Any modern browser (Chrome, Firefox) for accessing JSONPlaceholder documentation and registering for an OpenWeather API key.

3.3 Packages & Accounts

- No npm or pip packages are required for this guide — all requests are made via cURL and Postman.
- A free OpenWeather account is required to obtain an API key. Register at: openweathermap.org/api
- cURL is typically pre-installed on Linux. If missing, install it with: sudo apt install curl

3.4 Verifying Your Setup

Run the following commands in your terminal to confirm the required tools are installed:

```
# Check cURL is installed
curl --version

# Expected output (version may differ):
# curl 7.88.1 (x86_64-pc-linux-gnu) libcurl/7.88.1 ...

# Check your internet connection by pinging JSONPlaceholder
curl -s https://jsonplaceholder.typicode.com/users/1 | head -5

# Expected output: first few lines of a JSON user object
```

Section 4: Installation & Setup Instructions

This section walks through setting up every tool needed for the project. Follow each step in order.

4.1 Install cURL (if not already installed)

```
# Step 1: Update your package manager
sudo apt update

# Step 2: Install cURL
sudo apt install curl -y

# Step 3: Verify installation
curl --version
```

Tip: *Most Ubuntu and Debian distributions ship with cURL pre-installed. If curl --version returns a version number, you are ready to proceed.*

4.2 Install VS Code on Linux

1. Open your terminal.
2. Download the VS Code .deb package:

```
wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg
--dearmor > packages.microsoft.gpg
sudo install -o root -g root -m 644 packages.microsoft.gpg
/etc/apt/trusted.gpg.d/
sudo sh -c 'echo "deb [arch=amd64]
https://packages.microsoft.com/repos/vscode stable main" >
/etc/apt/sources.list.d/vscode.list'
sudo apt update
sudo apt install code -y
```

3. Launch VS Code from the terminal by typing: code

4.3 Install Postman on Linux

4. Visit https://www.postman.com/downloads/ and download the Linux (x64) tarball.
5. Extract and install:

```
# Extract the downloaded tarball (filename may vary)
tar -xzf Postman-linux-x64-*.tar.gz

# Move to /opt for system-wide access
sudo mv Postman /opt/Postman
```

```
# Create a desktop shortcut (optional)
sudo ln -s /opt/Postman/Postman /usr/local/bin/postman

# Launch Postman
postman &
```

4.4 Obtain an OpenWeather API Key

6. Go to https://openweathermap.org and click Sign Up.
7. Create a free account using your email address.
8. After confirming your email, navigate to: My Account > API Keys.
9. Copy your default API key — it is a 32-character alphanumeric string.
10. Store it safely (e.g., in a plain text file on your desktop). Never share it publicly.

> **Important:** *New API keys take up to 2 hours to activate after registration. If your first request returns a 401 error, wait and try again later.*

Section 5: Minimal Working Example

5.1 Example A — GET Request with cURL (JSONPlaceholder)

What This Example Does

This example fetches a single user record (user ID 1) from JSONPlaceholder's fake database. It demonstrates the most fundamental API operation: sending a GET request and reading the JSON response.

The Command

```
# Send a GET request to fetch user with ID 1
# -s: silent mode (hides progress meter)
# The URL is the endpoint: /users/1 means 'give me user number 1'
curl -s https://jsonplaceholder.typicode.com/users/1
```

Expected Output

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "city": "Gwenborough",
    "zipcode": "92998-3874"
  },
  "phone": "1-770-736-0988 x56442",
```

```
  "website": "hildegard.org"
}
```

What to Notice

- The response is JSON — key-value pairs wrapped in curly braces {}.
- Nested objects (like address) allow structured, multi-level data in a single response.
- Changing /users/1 to /users/2 fetches a different user — the ID in the URL is the filter.
- No API key was needed — JSONPlaceholder is open and public.

5.2 Example B — POST Request with cURL (JSONPlaceholder)

What This Example Does

This example sends a POST request to create a new post record. Unlike GET which only reads, POST sends data to the server. This is the pattern used when submitting a form, creating an account, or saving a record.

The Command

```
# Send a POST request with JSON data in the body
# -X POST: specifies the HTTP method
# -H: sets a header (telling the server the body is JSON)
# -d: the data (body) to send
curl -s -X POST https://jsonplaceholder.typicode.com/posts \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "My First API Post",
    "body": "Learning REST APIs on Linux.",
    "userId": 1
  }'
```

Expected Output

```
{
  "id": 101,
  "title": "My First API Post",
  "body": "Learning REST APIs on Linux.",
  "userId": 1
}
```

What to Notice

- The server returned HTTP 201 Created — confirming the resource was made.
- The server assigned id: 101 automatically — you sent the data, the server managed the ID.
- The Content-Type header is required for POST — it tells the server how to read the body.
- JSONPlaceholder does not actually save this data permanently; it simulates the response.

5.3 Example C — Authenticated GET with OpenWeather API

What This Example Does

This example fetches live weather data for Nairobi, Kenya from the OpenWeather API. It introduces API key authentication and query parameters — two concepts absent from JSONPlaceholder but essential for real-world APIs.

The Command

```
# Replace YOUR_API_KEY with your actual OpenWeather key
# q=Nairobi: the city to look up (query parameter)
# units=metric: return temperature in Celsius
# appid: your authentication key
curl -s
'https://api.openweathermap.org/data/2.5/weather?q=Nairobi&units=metric&a
ppid=YOUR_API_KEY'
```

Expected Output (Abbreviated)

```
{
  "name": "Nairobi",
  "main": {
    "temp": 22.4,
    "feels_like": 21.8,
    "humidity": 68
  },
  "weather": [{ "description": "broken clouds" }],
  "wind": { "speed": 3.6 }
}
```

What to Notice

- Query parameters follow a ? and are separated by &. They filter and customize the response.
- The appid parameter carries the API key — remove it and you get a 401 Unauthorized error.
- Changing units=metric to units=imperial returns Fahrenheit instead — parameters control output.
- The data is live and real — this is the same data a weather app on your phone would display.

Section 6: AI Prompt Journal

Throughout this project, AI (Claude by Anthropic) was used as a learning and problem-solving tool. The following journal documents each prompt submitted, a summary of the AI's response, and an evaluation of how helpful it was. This reflects honest, reflective use of AI — including cases where the response needed verification or refinement.

Prompt #1

Prompt Used: *"Give me a plain-English explanation of what a REST API is, as if I have never heard the term before."*

AI Response Summary: The AI provided a clear analogy comparing an API to a waiter in a restaurant — the client orders, the waiter (API) communicates with the kitchen (server), and brings back the result. It then explained HTTP methods (GET, POST, PUT, DELETE) with simple one-sentence definitions for each. No jargon was used without explanation.

Helpfulness Rating: Very helpful (5/5). The analogy made the concept immediately intuitive and gave a mental model to build on throughout the project. The response was verified against the MDN Web Docs definition of REST, which aligned closely.

Prompt #2

Prompt Used: *"Show me a cURL command to make a GET request on Linux, and explain each part of the command."*

AI Response Summary: The AI provided a complete cURL command with inline comments explaining each flag: -s for silent mode, the URL structure, and how to read the JSON output. It also showed a second variation using -v (verbose mode) for debugging, which displays headers alongside the response body.

Helpfulness Rating: Helpful (4/5). The command worked immediately on the first attempt. The verbose flag tip proved useful in Section 7 when debugging a 401 error — it revealed the missing API key in the response headers.

Prompt #3

Prompt Used: *"What is the difference between a query parameter and a path parameter in a REST API?"*

AI Response Summary: The AI explained that path parameters are part of the URL itself (e.g., /users/1 where 1 identifies a specific user), while query parameters are appended after a ? and act as optional filters (e.g., ?units=metric). It included examples of both patterns and noted when each is conventionally used.

Helpfulness Rating: Very helpful (5/5). This clarified why the OpenWeather API used ?q=Nairobi rather than /weather/Nairobi. Cross-referenced with the OpenWeather API docs to confirm accuracy.

Prompt #4

Prompt Used: *"I received a 401 Unauthorized error from the OpenWeather API even though I included my API key. What are the common causes?"*

AI Response Summary: The AI listed four common causes: (1) the key was copied with an extra space, (2) the key had not yet activated — new keys take up to 2 hours, (3) the parameter name was misspelled (appid not api_key), (4) the free plan did not cover the endpoint. It recommended using curl -v to inspect the full request.

Helpfulness Rating: Extremely helpful (5/5). Cause number 2 — the activation delay — was the actual reason for the error. Matched the guidance in the OpenWeather FAQ exactly.

Prompt #5

Prompt Used: *"Can you review this cURL POST command and tell me if there are any errors in it?" [command pasted inline]*

AI Response Summary: The AI identified that single quotes inside the JSON body were conflicting with the outer single-quoted -d argument in bash. It suggested escaping the inner quotes or switching to double quotes for the outer argument. It provided a corrected version of the command.

Helpfulness Rating: Helpful (4/5). The fix worked correctly. A reminder that AI is useful for both API learning and general terminal debugging.

> **Note:** *All AI responses were cross-referenced against official documentation (MDN Web Docs, JSONPlaceholder, OpenWeather API reference) before being accepted. AI accelerated learning but did not replace verification.*

Section 7: Common Issues & Fixes

The following issues were encountered during the project. Each is documented with the exact error or symptom observed and the resolution applied.

Issue Encountered	Error / Symptom	Resolution
cURL not found on system	bash: curl: command not found	Installed cURL via: sudo apt install curl -y. Confirmed with curl --version after install.
OpenWeather 401 on first request	{ "cod": 401, "message": "Invalid API key" }	API key had not yet been activated. Waited 2 hours after registration and retried. The request succeeded.
POST body not sent correctly	Server returned default empty response	Single quotes in bash conflicted with -d argument. Switched to double quotes with escaped inner quotes.
JSON output was unformatted	All response text on one line	Piped cURL output to Python: curl -s [url] | python3 -m json.tool for pretty-printed JSON.
Wrong units in OpenWeather	Temp returned as 295.4 (Kelvin)	The units=metric parameter was omitted. Added &units=metric to the URL to return Celsius.
Postman SSL error on first launch	Error: SSL certificate expired	Opened Postman Settings > General > SSL certificate verification and toggled it off for local testing.

7.1 Useful Debugging Tip: Verbose Mode

When a request fails and the error message is unclear, adding the -v flag to cURL shows the full request and response headers — including which URL was called, what headers were sent, and the exact HTTP status code returned by the server:

```
# -v shows full request/response headers for debugging
curl -v https://jsonplaceholder.typicode.com/users/1

# Look for lines starting with > (sent) and < (received)
# > GET /users/1 HTTP/1.1
# < HTTP/1.1 200 OK
```

7.2 Helpful Resources for Common Issues

- cURL documentation: https://curl.se/docs/manpage.html
- OpenWeather API FAQ: https://openweathermap.org/faq
- JSONPlaceholder docs: https://jsonplaceholder.typicode.com/guide
- Stack Overflow — curl tag: https://stackoverflow.com/questions/tagged/curl
- HTTP status codes reference: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

Section 8: References

The following sources were consulted during this project and are recommended for further reading.

8.1 Official Documentation

- JSONPlaceholder — Free fake REST API for testing: https://jsonplaceholder.typicode.com
- OpenWeather API — Current weather data: https://openweathermap.org/api/one-call-3
- MDN Web Docs — HTTP Methods: https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods
- MDN Web Docs — HTTP Status Codes: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
- cURL Man Page: https://curl.se/docs/manpage.html
- Postman Learning Centre: https://learning.postman.com

8.2 Video Resources

- "What is a REST API?" — Fireship (YouTube): https://www.youtube.com/watch?v=-MTSQjw5DrM
- "REST API Crash Course" — Traversy Media (YouTube): https://www.youtube.com/watch?v=Q-BpqyOT3a8
- "Using cURL to Test REST APIs" — TechWorld with Nana (YouTube): https://www.youtube.com/watch?v=7YcW25PHnAA

8.3 Helpful Articles & Blog Posts

- "REST API Tutorial" — restfulapi.net: https://restfulapi.net
- "What is JSON?" — W3Schools: https://www.w3schools.com/js/js_json_intro.asp
- "How to Use cURL in Linux" — linuxize.com: https://linuxize.com/post/curl-command-examples

- "Understanding API Authentication" — Postman Blog: https://blog.postman.com/what-is-api-authentication

8.4 AI Tool Used

- Claude by Anthropic (claude.ai) — Used as a learning tutor, concept explainer, and debugging aid throughout this project. All AI-generated explanations were verified against the official documentation listed above.

End of Guide — REST APIs: A Beginner's Guide | March 2026